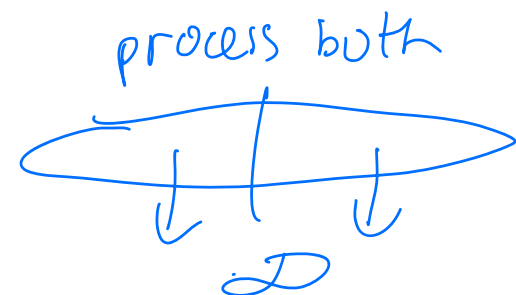


Divide and conquer

IRL



Problem flavors

- Binary search on array
- Binary parametric search
- Divide, process each, then merge

↪ merge sort

Binary search on array

FindFirstB

- Find the index of the first B given the array is of the form AA BBB . . .
- Or return -1 if B doesn't exist.

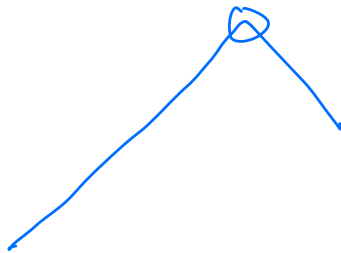
Examples

Input	Output
^{0 1 2} <u>AABBBB</u>	2
<u>AAAA</u>	-1
<u>B</u> BBB	0

1 3 5 7 10 13
find 5
 $A = < 5$
 $B = \geq 5$

Reduction example

- Binary search. Find target in sorted array, or return -1 if not found.
- Find peak element. See homework!



1 3 5 10 7 3

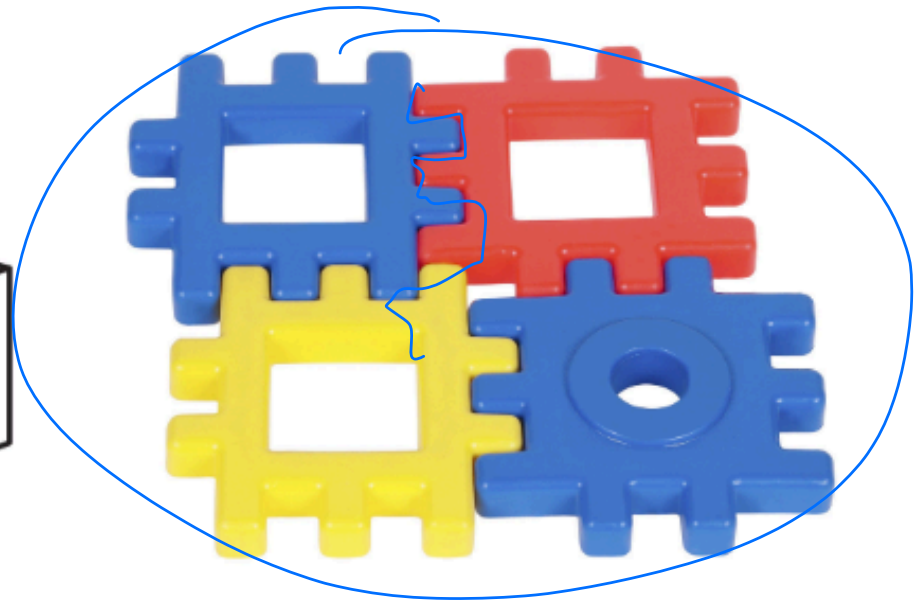
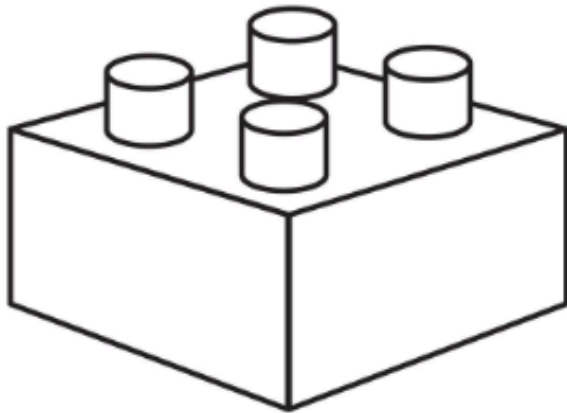
Implementing FindFirstB: Function specification

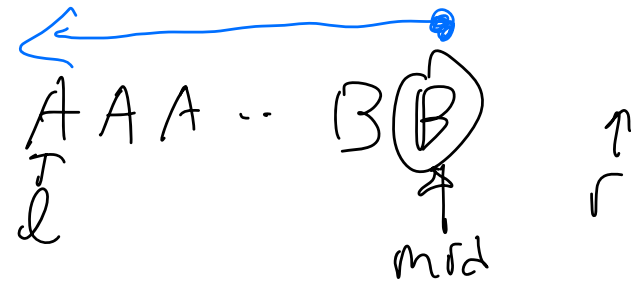
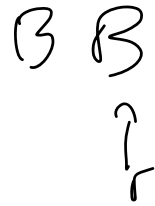
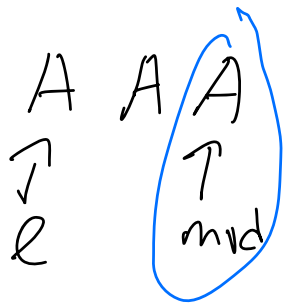
`findFirstB(char[] arr, int l, int r)`

- **Def 1: End inclusive.** Find the first 'B' in `arr[l..r]`, else return -1
- **Def 2: End exclusive.** Find the first 'B' in `arr[l..r-1]`, else return r

```
arr = "AABBB"  
findFirstBEndInclusive(arr, 0, 2) = "Search inside AAB"  
findFirstBEndExclusive(arr, 0, 2) = "Search inside AA"
```

Why be so specific?





Def 1: End inclusive

Function specification: TODO(1) return idx of -- first B in
 findFirstB(arr, l, r):
 # Base case. TODO(4) arr[l, r]

mid = (l + r) // 2

if arr[mid] == 'B':

TODO(2)

else:

TODO(3)

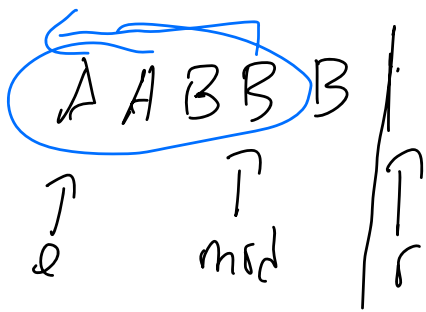
find First B (arr, ?, ?)
 find First B (?, ?) l mid

mid+1 r

Def 1: End inclusive

```
# find first B in arr[l..r] else -1
findFirstB(arr, l, r):
    # Base case
    if l > r: return -1
    if l == r: return l if arr[l] == 'B' else -1

    mid = (l + r) // 2
    if arr[mid] == 'B':
        return findFirstB(arr, l, mid)
    else:
        return findFirstB(arr, mid + 1, r)
```



Def 2: End exclusive

```
findFirstB("AA", 0, 2) = 2 # If not found return r
findFirstB("AB", 0, 2) = 1
```

Function specification: TODO(1) returns idx of first 'B' in
 arr [l, r-1]

```
findFirstB(arr, l, r):
    # Base case. TODO(4)
```

```
mid = (l + r) // 2
```

```
if arr[mid] == 'B':
```

```
    # TODO(2)
```

```
else: = "A"
```

```
    # TODO(3)
```

findFirstB(l, ?)

findFirstB(mid+1, r)

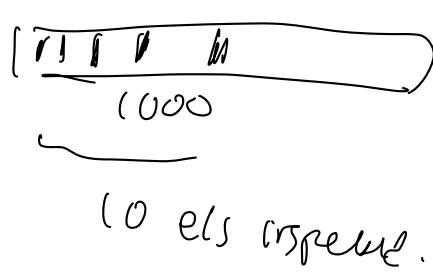
Def 2: End exclusive

```
# find first B in arr[l..r-1], else r
findFirstB(arr, l, r):
    # Base case
    if l == r: return r

    mid = (l + r) // 2
    if arr[mid] == 'B':
        return findFirstB(arr, l, mid)
    else:
        return findFirstB(arr, mid + 1, r)
```

Handwritten note: mid + 1

1. $O(\log n)$



$$\frac{\log n}{2} = O(\log n)$$

2. $O(n \log n)$

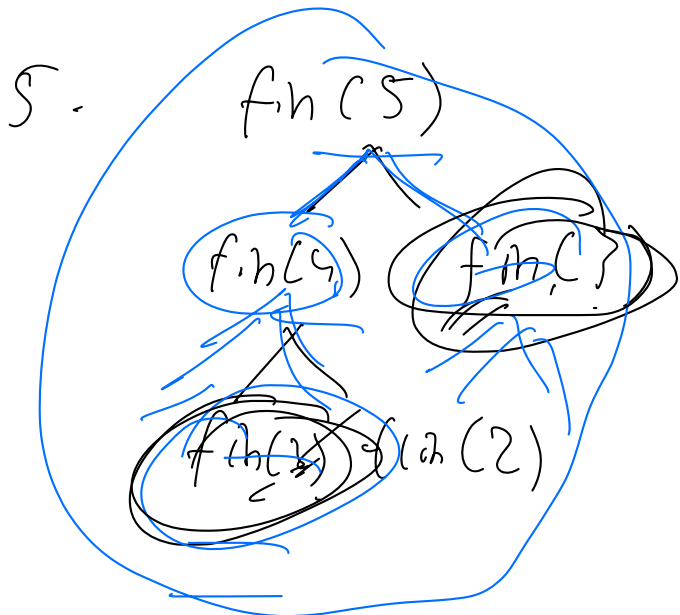
3. for $i = 1 \rightarrow n$
 $j = 1 \rightarrow n$

n^2 $O(n^2)$

Diagram showing nested loops for $i = 1$ to n and $j = 1$ to n , with a bracket indicating $n \times n = n^2$.

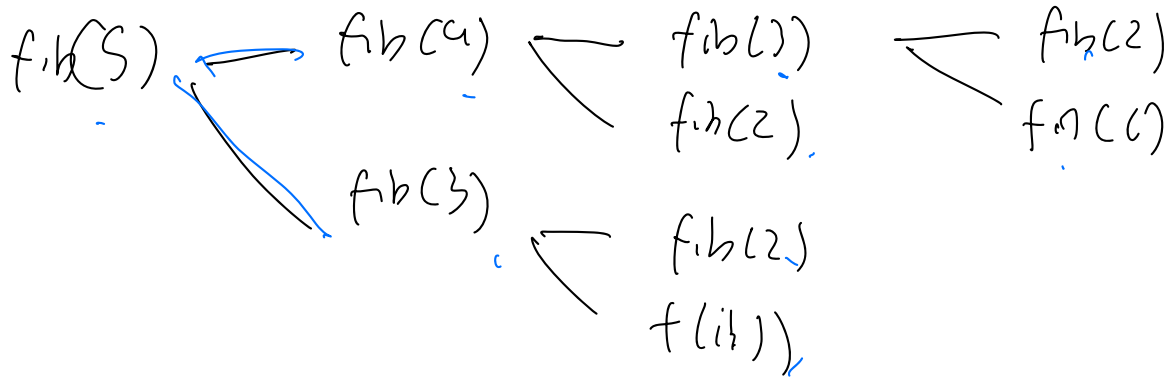
4. for $i = 0 \rightarrow n-1$
 $j = 0 \rightarrow i-1$

Diagram showing the sum of integers from 1 to n , with a bracket indicating $\frac{n(n-1)}{2} = O(n^2)$.



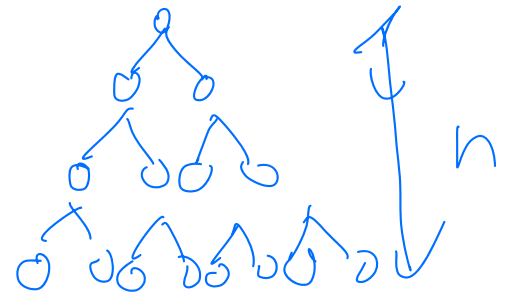
Very bad!!

$O(2^n)$



9 calls for $\text{fib}(5)$

$O(n)$



6. $\text{fib}(10) \rightarrow 1$

$\text{fib}(9) \rightarrow 1$

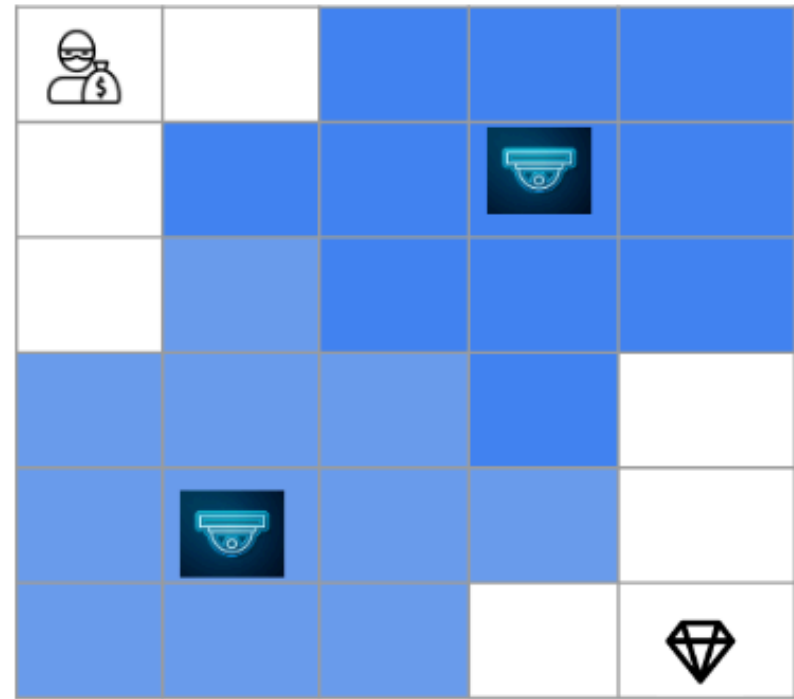
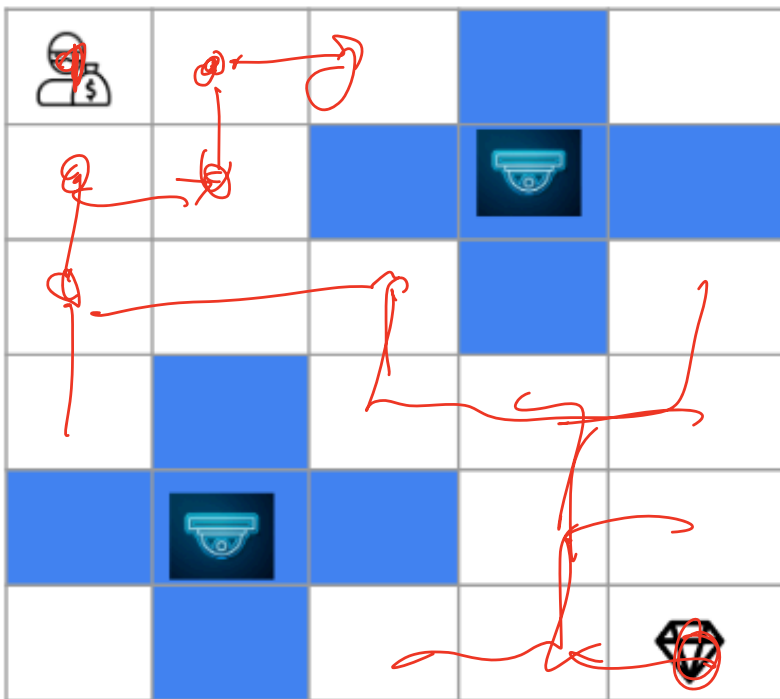
$\text{fib}(8) \rightarrow \dots$

1

$O(n)$

$O(n)$

Parametric search: Stop the thief



BFS/ DFS to check

monotonicity

What to binary search on?

Parameter (sensor power)	Is thief stopped?
0	No
1	No
2	Yes
3	Yes
...	...

Parametric search

- Define bool isFeasible(parameter)
- Check: IsFeasible monotone?
- Binary search parameter

param
↑
→ is thief stopped (sensor power)

param →
no
no
no
yes
yes

000
101

$i = \text{sensor power}$	1	2	3	4	5	...
$\text{arr}[i] = \text{thief stopped?}$	F	F	F	T	T	...

Pseudocode

```

findFirstB(
    arr[i] = isThiefStopped(sensor coverage = i)
    l = 0
    r = grid size)
  
```

① $\text{isThiefStopped}(\text{int sensor power})$

- BFS

- $O(?) \rightarrow O(n^2)$

② call $\text{isThiefStopped} \rightarrow O(\log s)$

③ in total = $O(n^2 \log s)$

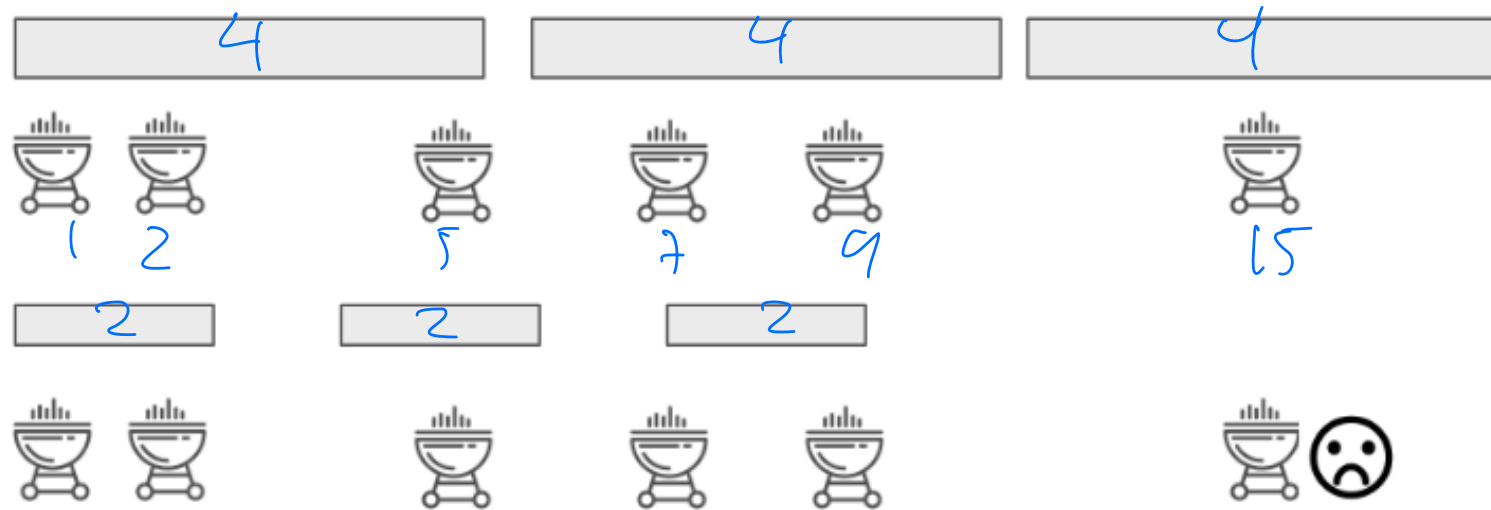
$n \times n$ grid

$S = \text{max sensor power}$

- ① bool isCovered (int umbrella size)
- ② is it monotonic? size ↑ no no no .. yes - you see.
- ③ isCovered (int umbrella size, int[] grill locations) → bool

Umbrellas for bbq → $O(n)$

greedy soln = just place leftmost & repeat "barely"



- grill positions fixed
- # umbrellas fixed

Q: Input: int[] grill positions
int numUmbrella

Output: min umbrella width

Pseudocode

```
findFirstB(  
    arr[i] = isEveryoneCovered(umbrella size = i)  
    l = 0  
    r = field_length / num_umbrella)
```

can't binary search.



no $O(\log n)$:)

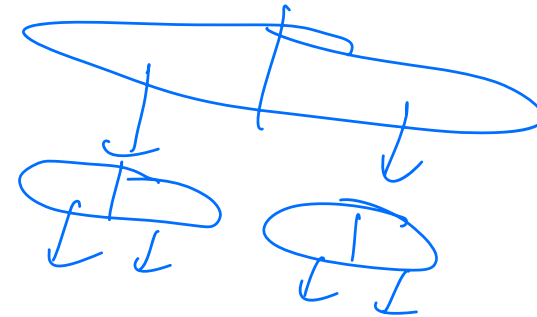
Non-monotonic? :(

Almost sorted

index	0	1	2	3	4	5	6
value	1	5	4	50	80	79	100



- ① find first B on array
- ② find first B on params — bha, href
- ③ merge sort

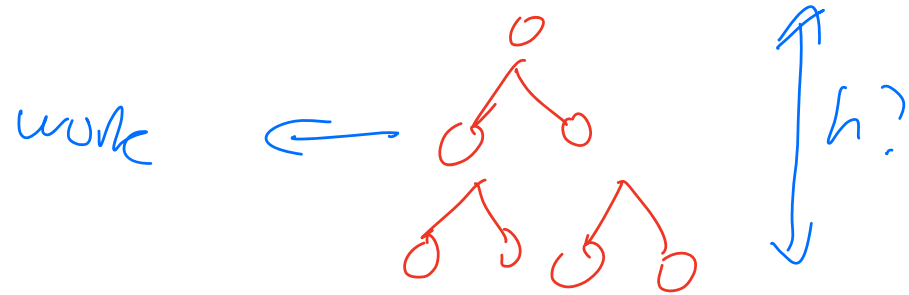


Divide, process each, merge

Merge sort

https://en.wikipedia.org/wiki/Merge_sort

<https://opensa-server.cs.vt.edu/embed/mergesortAV>



Runtime analysis

Questionnaire

- Recursion tree height
- Total work at each level
- Total work = sum of total work at each level

Binary search runtime analysis

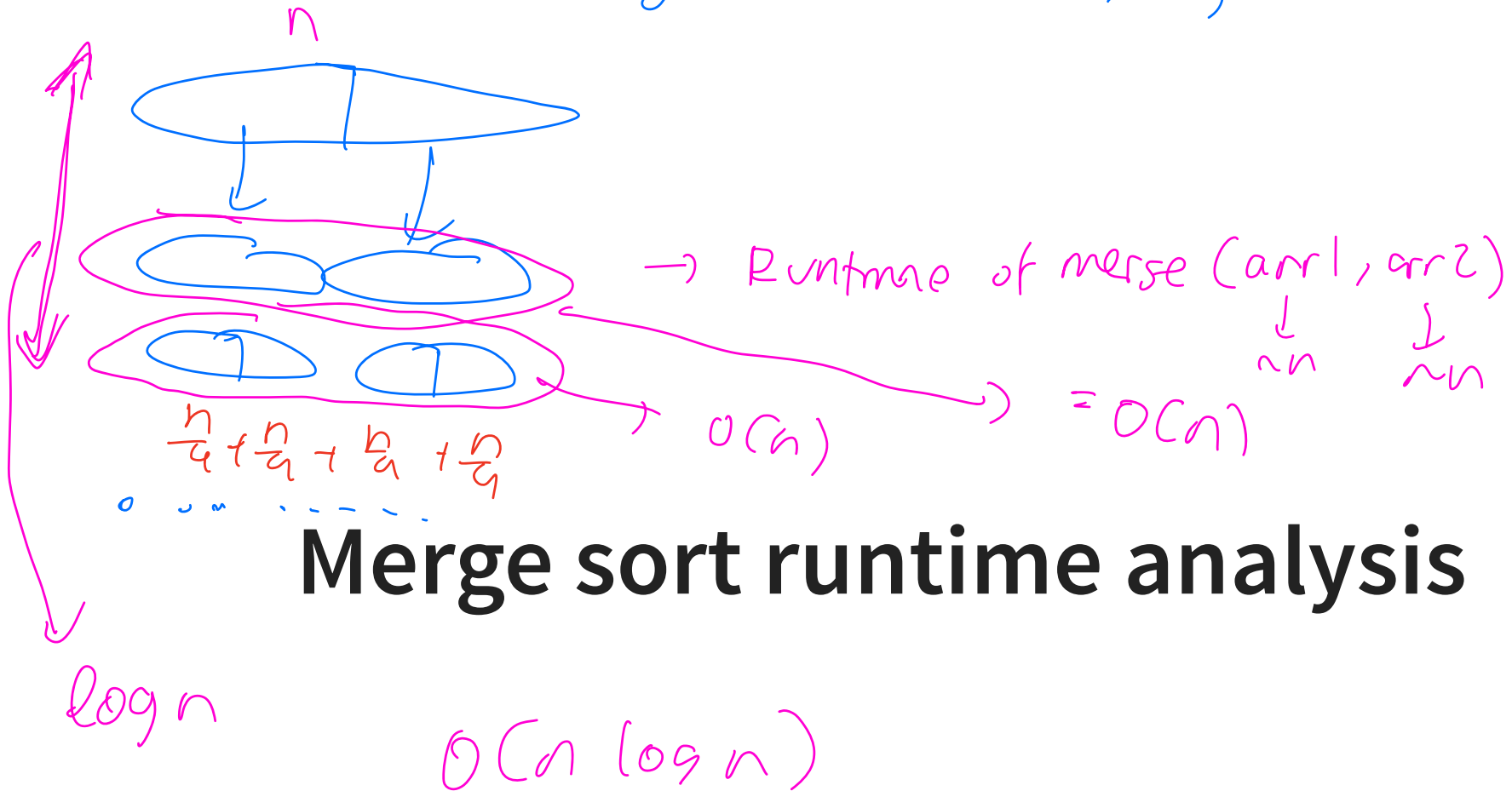
#1. #calls $\times$ work for each call
 $O(\log(S) \times N^2 \times S)$

Spot the thief runtime analysis

Assume: isThiefStopped(sensorPower) is $O(N^2 S)$

$O(\log S \times \checkmark)$

goal: $O(N \log n)$



Master theorem

$$T(n) = aT(n/b) + f(n)$$

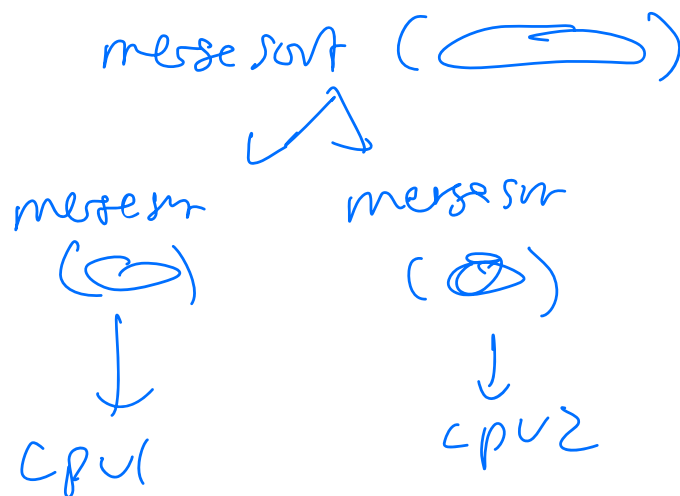
[Wikipedia](#)

no call stack
human

smaller mem footprint
faster

IRL

- Loops > recursion, if single threaded
- Parallelization. Divide and conquer preferred irl.



parallel merge sort
 $O(\log^3 n)$

Recap

